

TỰ HỌC STREAMLIT QUA CÁC CHỦ ĐỀ TOÁN HỌC



PHẦN 1
CƠ BẢN
TRẦN LÊ XUÂN HẠNH

ĐẮK LẮK

7 • 2024

CÁC BÀI HỌC

BÀI 1

Tạo ứng dụng tính bình phương một số (Trang 2-3)

BÀI 2

Tạo ứng dụng giải phương trình bậc nhất (Trang 4-6)

BÀI 3

Tạo ứng dụng vẽ đồ thị hàm số (Trang 7-9)

BÀI 4

Tạo ứng dụng tính toán thống kê đơn giản (Trang 10-12)

BÀI 5

Tạo ứng dụng chuyển đổi đơn vị đo lường (Trang 13-14)

BÀI 6

Tạo ứng dụng trực quan hóa dữ liệu đơn giản (Trang 15-17)

BÀI 7

Tạo ứng dụng giải phương trình và vẽ đồ thị hàm số (Trang 18-21)

BÀI 8

Tạo ứng dụng mô phỏng và tính toán xác suất (Trang 22-25)

Bài học 1

Tạo ứng dụng tính bình phương một số

MỤC TIÊU

- Hiểu cách tạo một ứng dụng Streamlit đơn giản
- Học cách sử dụng các thành phần cơ bản như tiêu đề, nhập liệu và hiển thị kết quả

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit (cài đặt bằng lệnh: `pip install streamlit`)

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st

st.title("Ứng dụng tính bình phương")

number = st.number_input("Nhập một số:", value=0)

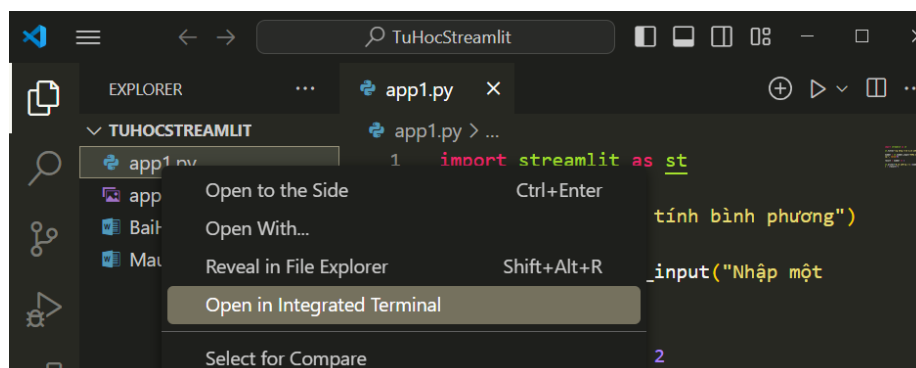
result = number ** 2

st.write(f"Bình phương của {number} là: {result}")
```

GIẢI THÍCH

1. `import streamlit as st`: Nhập thư viện Streamlit và đặt tên ngắn gọn là "st".
2. `st.title("Ứng dụng tính bình phương")`: Tạo tiêu đề cho ứng dụng.
3. `number = st.number_input("Nhập một số:", value=0)`: Tạo một ô nhập số, với giá trị mặc định là 0.
4. `result = number ** 2`: Tính bình phương của số đã nhập.
5. `st.write(f"Bình phương của {number} là: {result}")`: Hiển thị kết quả tính toán.

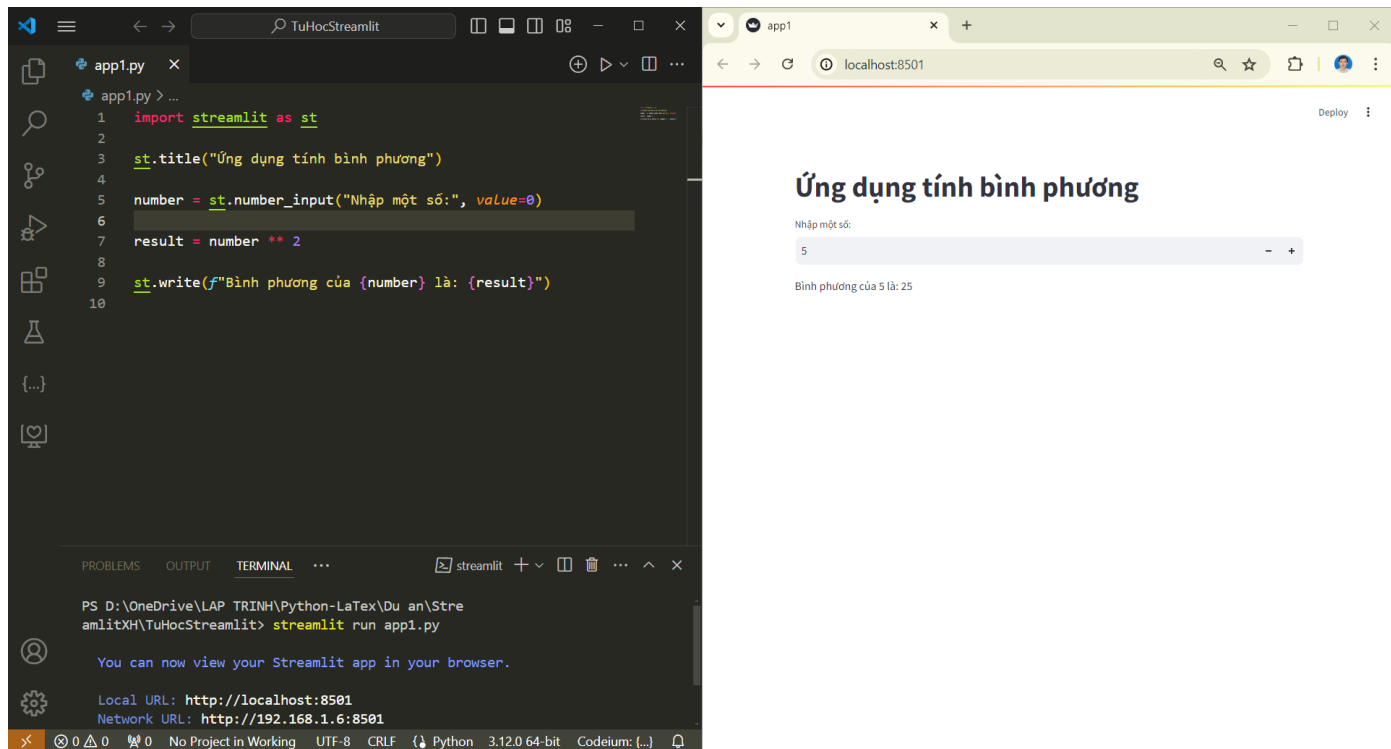
MINH HỌA



Vào TERMINAL gõ `streamlit run app1.py` (chú ý đường dẫn đến thư mục chứa file `app1.py`)

Ví dụ:

D:\OneDrive\LAP TRINH\Python-LaTex\Du an\StreamlitXH\TuHocStreamlit> streamlit run app1.py



BÀI TẬP

Hãy tạo một ứng dụng tính căn bậc hai của một số. Sử dụng hàm `math.sqrt()` từ thư viện `math` để tính căn bậc hai.

KẾT LUẬN

Trong bài học này, chúng ta đã tạo một ứng dụng Streamlit đơn giản để tính bình phương của một số. Chúng ta đã học cách sử dụng các thành phần cơ bản như tiêu đề (`st.title()`), nhập liệu (`st.number_input()`), và hiển thị kết quả (`st.write()`).

Bài học 2

Tạo ứng dụng giải phương trình bậc nhất

MỤC TIÊU

- Học cách sử dụng nhiều trường nhập liệu
- Thực hiện tính toán phức tạp hơn
- Hiển thị kết quả có điều kiện

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st

st.title("Ứng dụng giải phương trình bậc nhất")

st.write("Phương trình có dạng:  $ax + b = 0$ ")

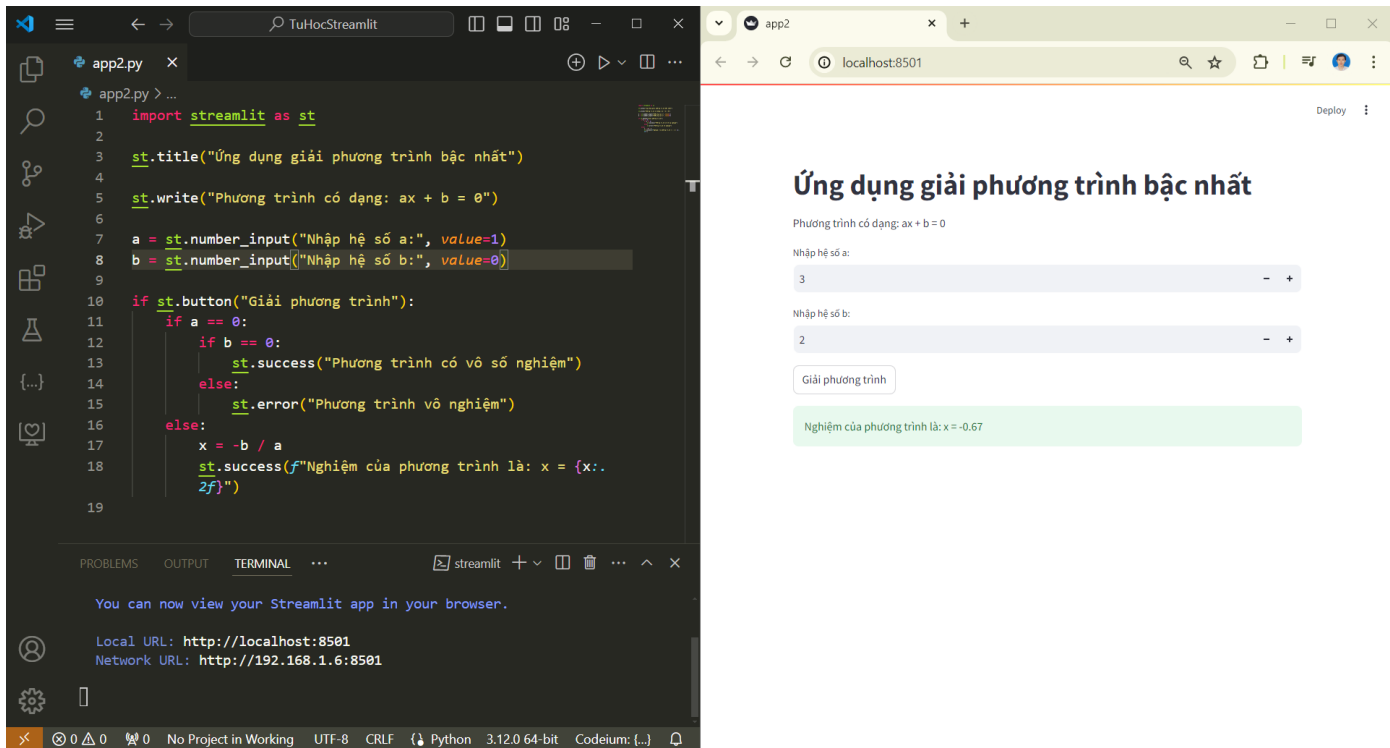
a = st.number_input("Nhập hệ số a:", value=1)
b = st.number_input("Nhập hệ số b:", value=0)

if st.button("Giải phương trình"):
    if a == 0:
        if b == 0:
            st.success("Phương trình có vô số nghiệm")
        else:
            st.error("Phương trình vô nghiệm")
    else:
        x = -b / a
        st.success(f"Nghiệm của phương trình là:  $x = {x:.2f}$ ")
```

GIẢI THÍCH

- 1-2. Nhập thư viện và tạo tiêu đề (giống bài trước).
3. `st.write("Phương trình có dạng:  $ax + b = 0$ ")`: Hiển thị dạng phương trình.
- 4-5. Tạo hai ô nhập số cho hệ số a và b.
6. `if st.button("Giải phương trình")`: Tạo nút "Giải phương trình" và kiểm tra khi nút được nhấn.
- 7-14. Thực hiện giải phương trình:
Nếu $a = 0$, kiểm tra b để xác định vô số nghiệm hoặc vô nghiệm.
Nếu $a \neq 0$, tính nghiệm $x = -b/a$.
15. Hiển thị kết quả bằng `st.success()` hoặc `st.error()`.

MINH HỌA

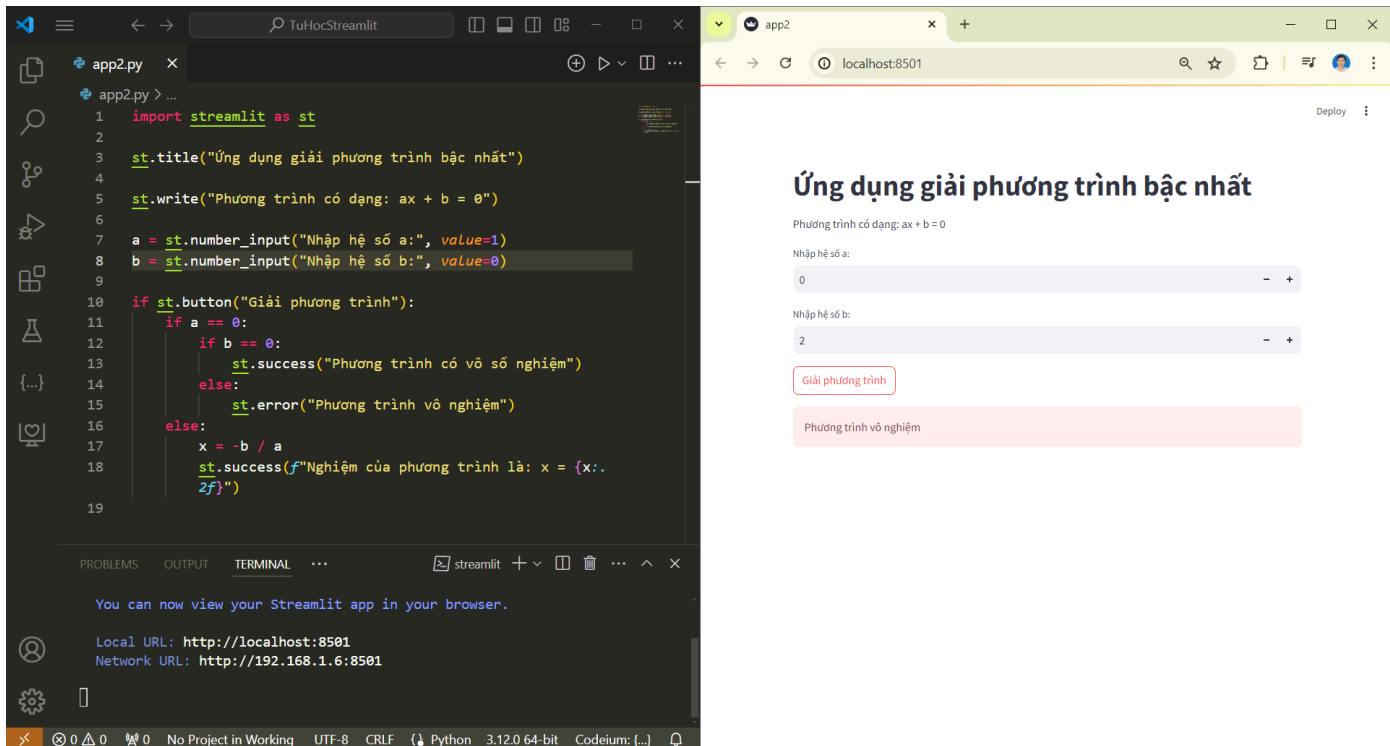


The screenshot shows a Streamlit application running in a browser. The app is titled "Ứng dụng giải phương trình bậc nhất". It displays the equation $ax + b = 0$ and the inputs $a = 3$ and $b = 2$. The result is $x = -0.67$.

```
1 import streamlit as st
2
3 st.title("Ứng dụng giải phương trình bậc nhất")
4
5 st.write("Phương trình có dạng: ax + b = 0")
6
7 a = st.number_input("Nhập hệ số a:", value=1)
8 b = st.number_input("Nhập hệ số b:", value=0)
9
10 if st.button("Giải phương trình"):
11     if a == 0:
12         if b == 0:
13             st.success("Phương trình có vô số nghiệm")
14         else:
15             st.error("Phương trình vô nghiệm")
16     else:
17         x = -b / a
18         st.success(f"Nghiệm của phương trình là: x = {x:.2f}")
19
```

Local URL: <http://localhost:8501>
Network URL: <http://192.168.1.6:8501>

Trường hợp có 1 nghiệm duy nhất

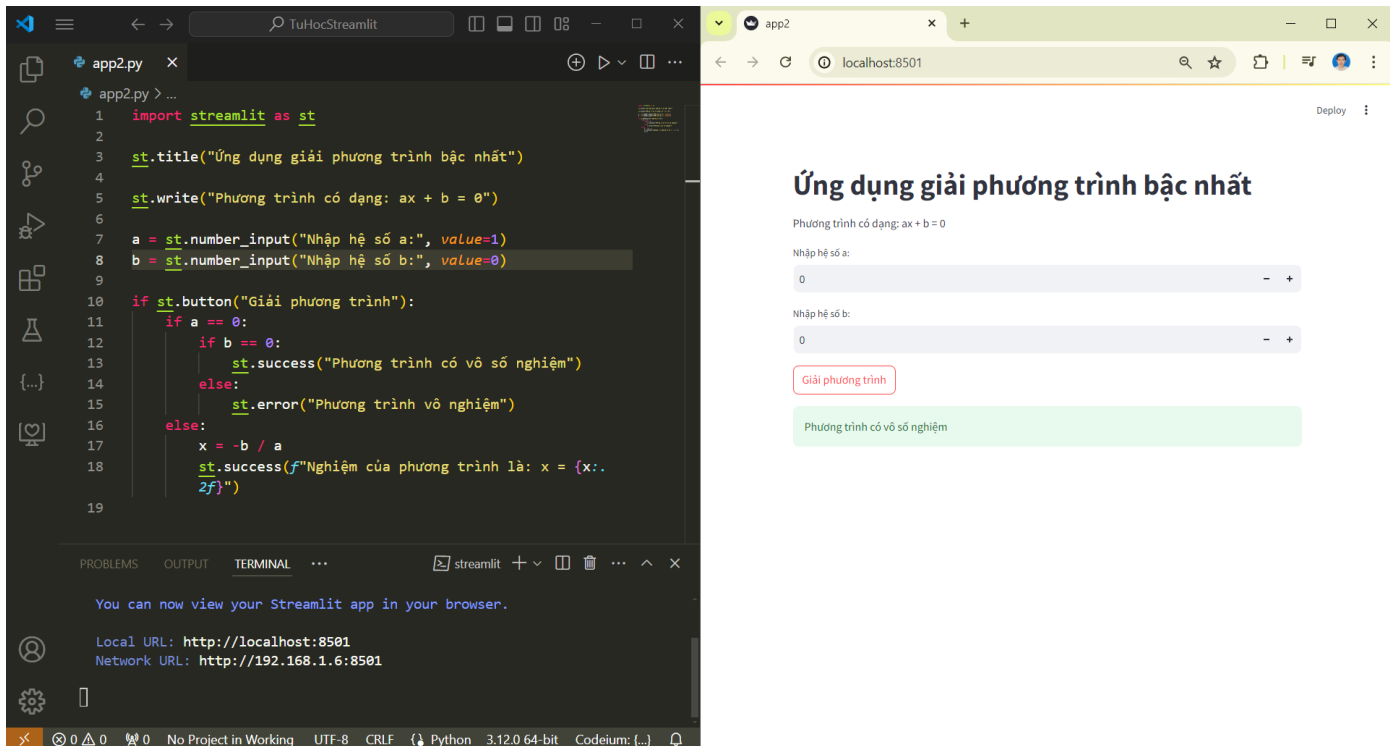


The screenshot shows the same Streamlit application, but with inputs $a = 0$ and $b = 2$. The result is "Phương trình vô nghiệm".

```
1 import streamlit as st
2
3 st.title("Ứng dụng giải phương trình bậc nhất")
4
5 st.write("Phương trình có dạng: ax + b = 0")
6
7 a = st.number_input("Nhập hệ số a:", value=1)
8 b = st.number_input("Nhập hệ số b:", value=0)
9
10 if st.button("Giải phương trình"):
11     if a == 0:
12         if b == 0:
13             st.success("Phương trình có vô số nghiệm")
14         else:
15             st.error("Phương trình vô nghiệm")
16     else:
17         x = -b / a
18         st.success(f"Nghiệm của phương trình là: x = {x:.2f}")
19
```

Local URL: <http://localhost:8501>
Network URL: <http://192.168.1.6:8501>

Trường hợp vô nghiệm



Trường hợp có vô số nghiệm

BÀI TẬP

Hãy tạo một ứng dụng giải phương trình bậc hai $ax^2 + bx + c = 0$. Nhớ xử lý các trường hợp đặc biệt ($a = 0$, $\Delta < 0$, $\Delta = 0$, $\Delta > 0$).

KẾT LUẬN

Trong bài học này, chúng ta đã tạo một ứng dụng phức tạp hơn để giải phương trình bậc nhất. Chúng ta đã học cách sử dụng nhiều trường nhập liệu, tạo nút với `st.button()`, và hiển thị kết quả có điều kiện bằng `st.success()` và `st.error()`.

Bài học 3

Tạo ứng dụng vẽ đồ thị hàm số

MỤC TIÊU

- Học cách tích hợp biểu đồ vào ứng dụng Streamlit
- Sử dụng thư viện matplotlib để vẽ đồ thị
- Tạo widget để tương tác với đồ thị

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit
- Thư viện matplotlib (cài đặt bằng lệnh: `pip install matplotlib`)
- Thư viện numpy (cài đặt bằng lệnh: `pip install numpy`)

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st
import matplotlib.pyplot as plt
import numpy as np

st.title("Ứng dụng vẽ đồ thị hàm số")

def f(x, a, b, c):
    return a * x**2 + b * x + c

a = st.slider("Hệ số a:", min_value=-5.0, max_value=5.0,
value=1.0, step=0.1)
b = st.slider("Hệ số b:", min_value=-5.0, max_value=5.0,
value=0.0, step=0.1)
c = st.slider("Hệ số c:", min_value=-5.0, max_value=5.0,
value=0.0, step=0.1)

x = np.linspace(-10, 10, 200)
y = f(x, a, b, c)

fig, ax = plt.subplots()
ax.plot(x, y)
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title(f'Đồ thị hàm số: y = {a}x^2 + {b}x + {c}')
ax.grid(True)

st.pyplot(fig)

st.write(f"Hàm số: y = {a}x^2 + {b}x + {c}")
```

GIẢI THÍCH

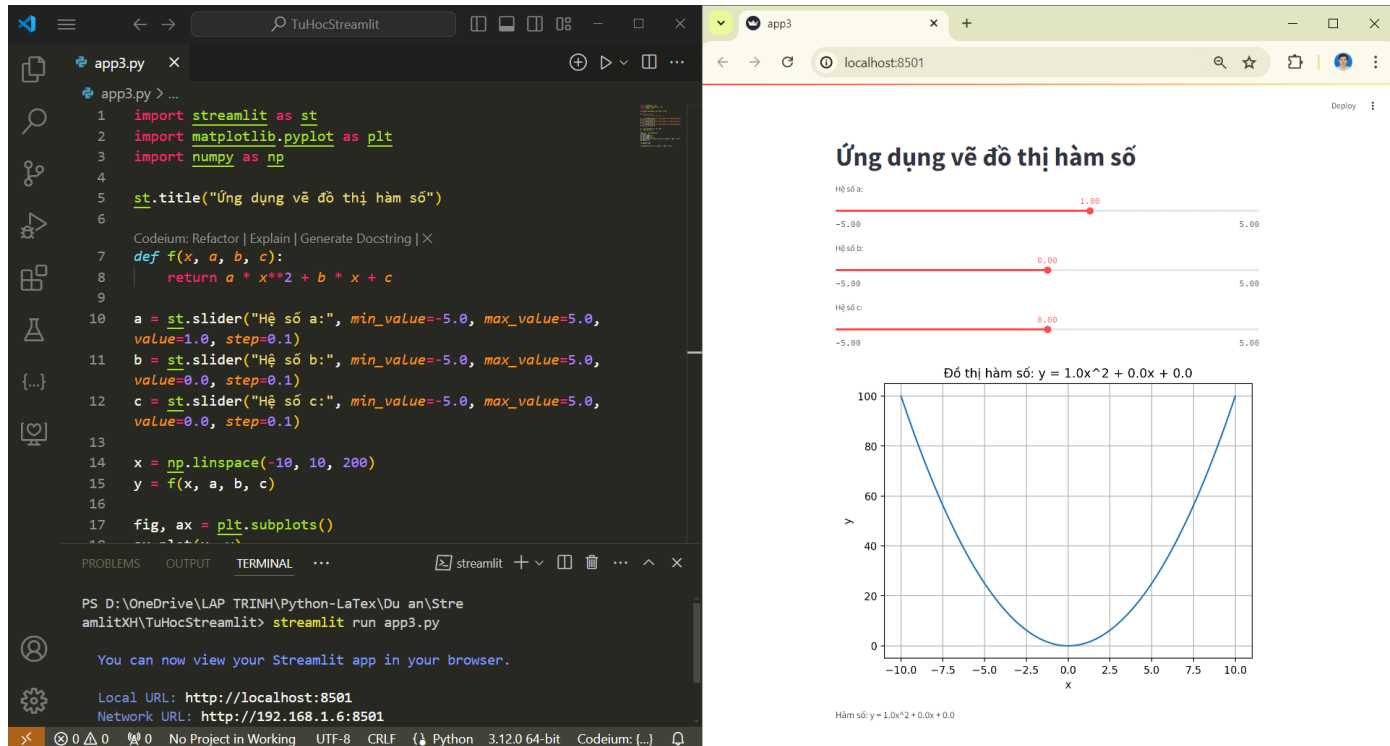
- 1-3. Nhập các thư viện cần thiết.
5. Tạo tiêu đề cho ứng dụng.
- 7-8. Định nghĩa hàm $f(x)$ là hàm bậc hai.
- 10-12. Tạo ba thanh trượt để người dùng có thể điều chỉnh các hệ số a, b, c .
- 14-15. Tạo mảng x và tính giá trị y tương ứng.
- 17-22. Tạo và cấu hình đồ thị matplotlib:

- Tạo đối tượng figure và axes
- Vẽ đồ thị
- Đặt nhãn cho trục x, y và tiêu đề
- Thêm lưới

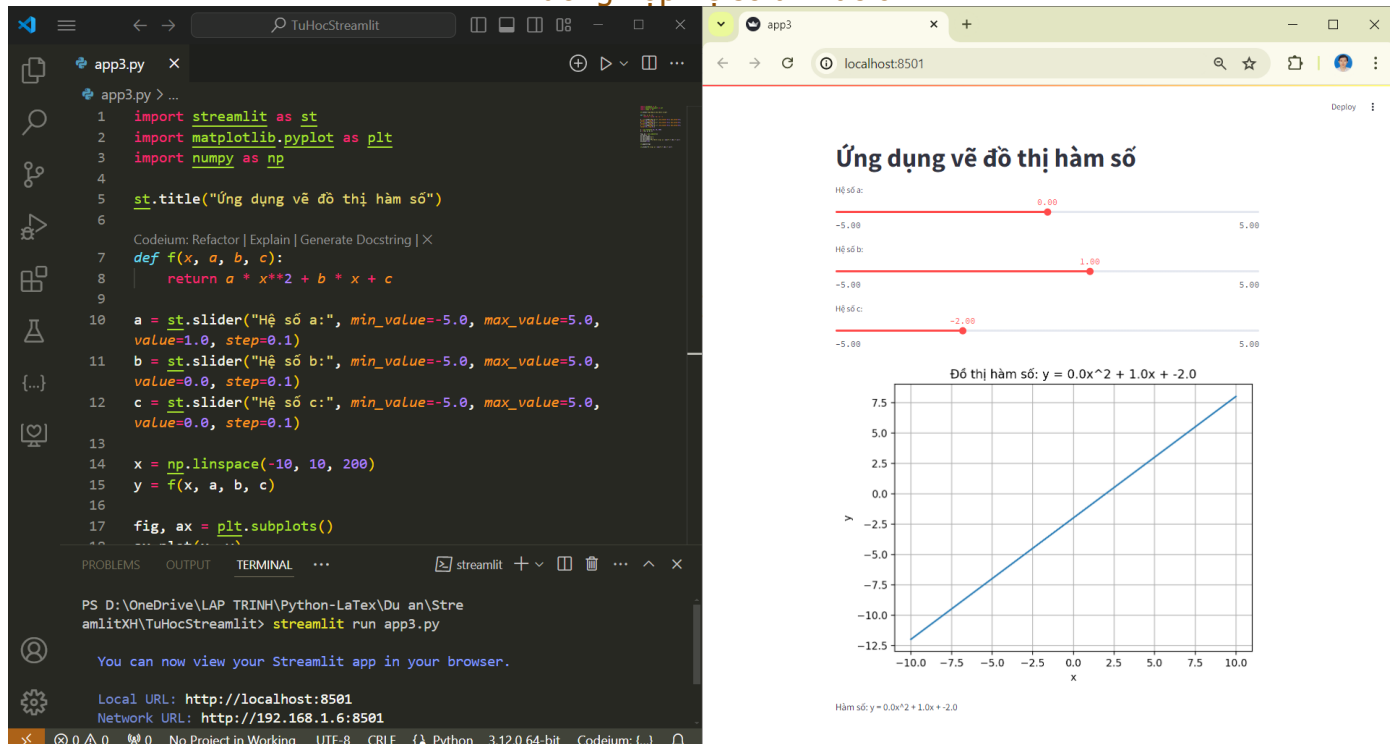
24. Hiện thị đồ thị trong ứng dụng Streamlit.

26. Hiện thị phương trình tương ứng.

MINH HỌA



Trường hợp hệ số a khác 0



Trường hợp hệ số a bằng 0

BÀI TẬP

Hãy tạo một ứng dụng vẽ đồ thị hàm sin và cos. Cho phép người dùng điều chỉnh biên độ và tần số của hàm.

Gợi ý:

1. Tạo dữ liệu cho đồ thị

```
x = np.linspace(0, 10, 1000)
y_sin = biendo_sin * np.sin(tanso_sin * x)
y_cos = biendo_cos * np.cos(tanso_cos * x)
```

2. Vẽ đồ thị

```
fig, ax = plt.subplots()
ax.plot(x, y_sin, Label=f'sin({tanso_sin}x) với biên độ {biendo_sin}')
ax.plot(x, y_cos, Label=f'cos({tanso_cos}x) với biên độ {biendo_cos}')
ax.legend()
```

KẾT LUẬN

Trong bài học này, chúng ta đã tạo một ứng dụng Streamlit để vẽ đồ thị hàm số bậc hai. Chúng ta đã học cách sử dụng thanh trượt ([st.slider\(\)](#)) để tạo widget tương tác, tích hợp matplotlib để vẽ đồ thị, và hiển thị đồ thị trong ứng dụng Streamlit bằng [st.pyplot\(\)](#).

Bài học 4

Tạo ứng dụng tính toán thống kê đơn giản

MỤC TIÊU

- Học cách sử dụng text area để nhập nhiều dữ liệu
- Xử lý chuỗi đầu vào và chuyển đổi thành dữ liệu số
- Thực hiện các phép tính thống kê cơ bản
- Sử dụng các cột trong Streamlit

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit
- Thư viện statistics (có sẵn trong Python standard library)

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st
import statistics as stats

st.title("Ứng dụng tính toán thống kê đơn giản")

data_input = st.text_area("Nhập các số, mỗi số trên một dòng:")

if st.button("Tính toán"):
    try:
        data = [float(x) for x in data_input.split('\n') if x.strip() != '']

        col1, col2 = st.columns(2)

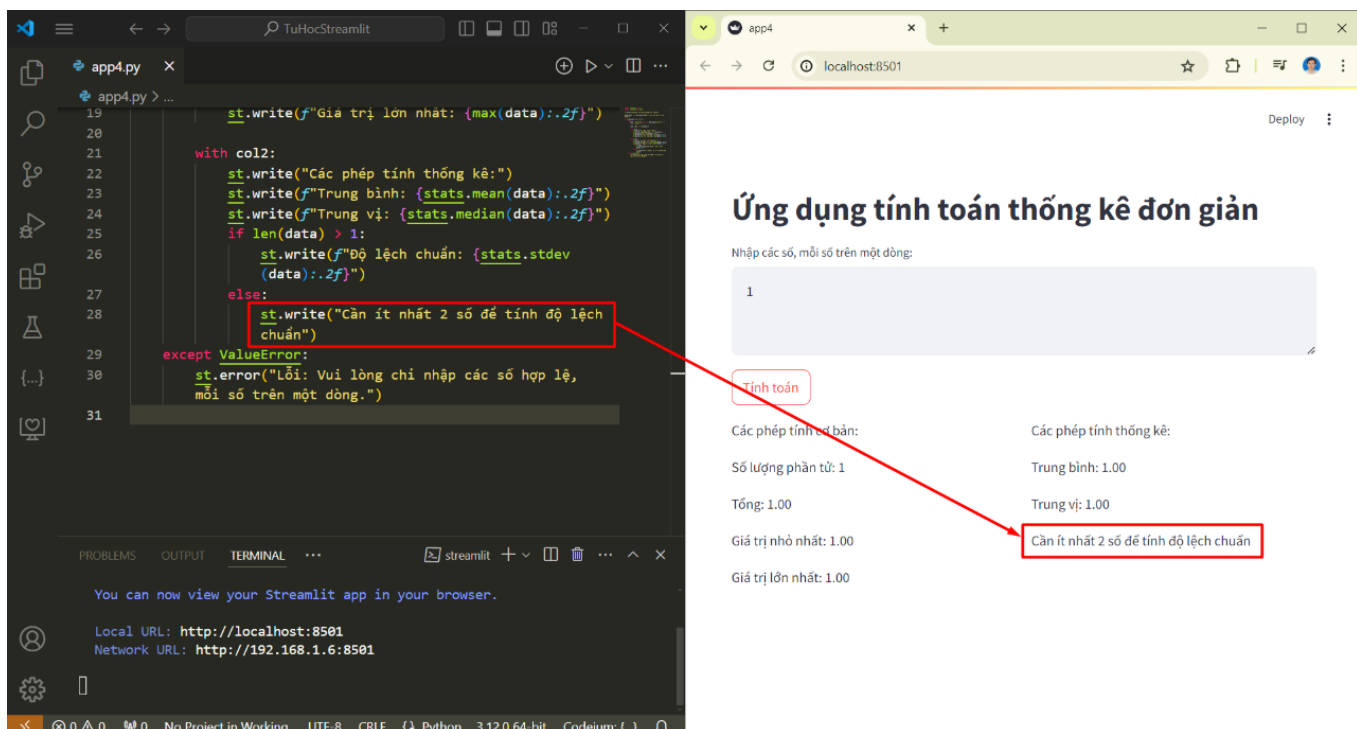
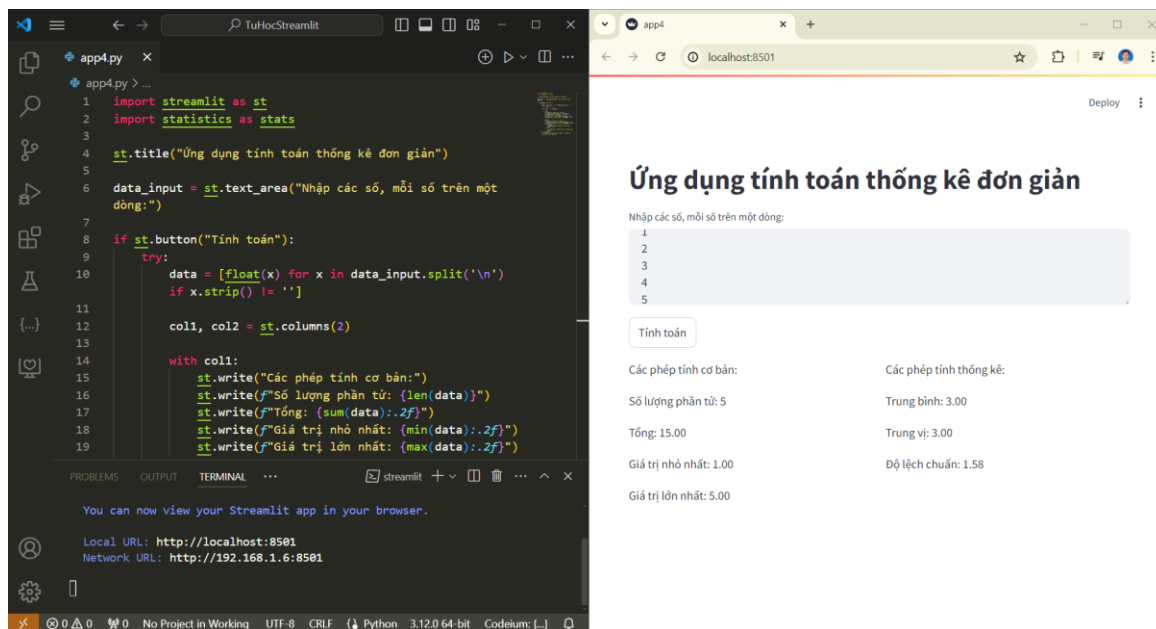
        with col1:
            st.write("Các phép tính cơ bản:")
            st.write(f"Số lượng phần tử: {len(data)}")
            st.write(f"Tổng: {sum(data):.2f}")
            st.write(f"Giá trị nhỏ nhất: {min(data):.2f}")
            st.write(f"Giá trị lớn nhất: {max(data):.2f}")

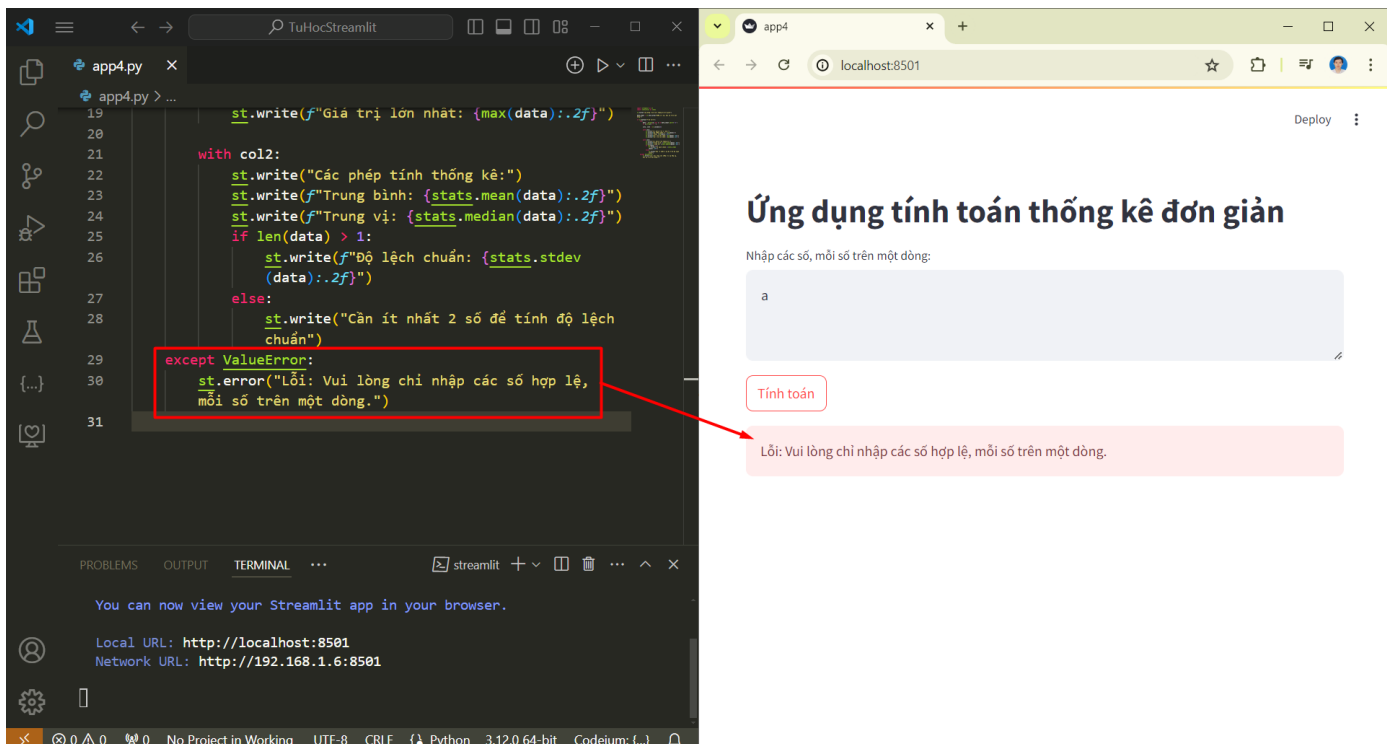
        with col2:
            st.write("Các phép tính thống kê:")
            st.write(f"Trung bình: {stats.mean(data):.2f}")
            st.write(f"Trung vị: {stats.median(data):.2f}")
            if len(data) > 1:
                st.write(f"Độ lệch chuẩn: {stats.stdev(data):.2f}")
            else:
                st.write("Cần ít nhất 2 số để tính độ lệch chuẩn")
        except ValueError:
            st.error("Lỗi: Vui lòng chỉ nhập các số hợp lệ, mỗi số trên một dòng.")
```

GIẢI THÍCH

- 1-2. Nhập các thư viện cần thiết.
3. Tạo tiêu đề cho ứng dụng.
4. Tạo một text area để nhập dữ liệu.
- 5-7. Tạo nút "Tính toán".
- 8-25. Xử lý dữ liệu và hiển thị kết quả khi nút được nhấn:
 - Chuyển đổi đầu vào thành danh sách các số thực.
 - Tạo hai cột để hiển thị kết quả.
 - Tính toán và hiển thị các thống kê cơ bản trong cột 1.
 - Tính toán và hiển thị các thống kê nâng cao trong cột 2.
 - Kiểm tra số lượng phần tử trước khi tính độ lệch chuẩn.
- 26-27. Xử lý lỗi nếu đầu vào không hợp lệ.

MINH HỌA





BÀI TẬP

Hãy mở rộng ứng dụng thống kê đơn giản bằng cách thêm các tính năng sau:

1. Tính và hiển thị các số tứ phân vị (Q1, Q2, Q3).
2. Tạo bảng phân bố tần số đơn giản.
3. Thêm tùy chọn để người dùng chọn hiển thị biểu đồ histogram của dữ liệu.

Gợi ý:

- Để tính số tứ phân vị, bạn có thể sử dụng hàm `statistics.quantiles()` với tham số `n=4`.
- Để tạo bảng phân bố tần số, bạn có thể sử dụng `pandas.cut()` để chia dữ liệu thành các khoảng và `pandas.value_counts()` để đếm số lượng phần tử trong mỗi khoảng.
- Để vẽ histogram, bạn có thể sử dụng `matplotlib.pyplot.hist()` và hiển thị bằng `st.pyplot()`.

Ví dụ về cách tính số tứ phân vị:

```
quartiles = stats.quantiles(data, n=4)
st.write(f"Q1 (25%): {quartiles[0]:.2f}")
st.write(f"Q2 (50%): {quartiles[1]:.2f}")
st.write(f"Q3 (75%): {quartiles[2]:.2f}")
```

KẾT LUẬN

Trong bài học này, chúng ta đã tạo một ứng dụng Streamlit để thực hiện các phép tính thống kê đơn giản. Chúng ta đã học cách sử dụng text area (`st.text_area()`) để nhập nhiều dữ liệu, xử lý chuỗi đầu vào, sử dụng thư viện statistics để tính toán, và tổ chức giao diện bằng cách sử dụng các cột (`st.columns()`).

Bài học 5

Tạo ứng dụng chuyển đổi đơn vị đo lường

MỤC TIÊU

- Học cách sử dụng selectbox để tạo menu thả xuống
- Sử dụng radio button để chọn hướng chuyển đổi
- Tạo ứng dụng có nhiều chức năng chuyển đổi

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st

st.title("Ứng dụng chuyển đổi đơn vị đo lường")

# Chọn loại chuyển đổi
conversion_type = st.selectbox(
    "Chọn loại chuyển đổi:",
    ("Độ dài", "Khối lượng", "Nhiệt độ")
)

# Chọn hướng chuyển đổi
direction = st.radio(
    "Chọn hướng chuyển đổi:",
    ("Từ đơn vị cơ bản", "Sang đơn vị cơ bản")
)

# Nhập giá trị cần chuyển đổi
value = st.number_input("Nhập giá trị cần chuyển đổi:",
value=0)

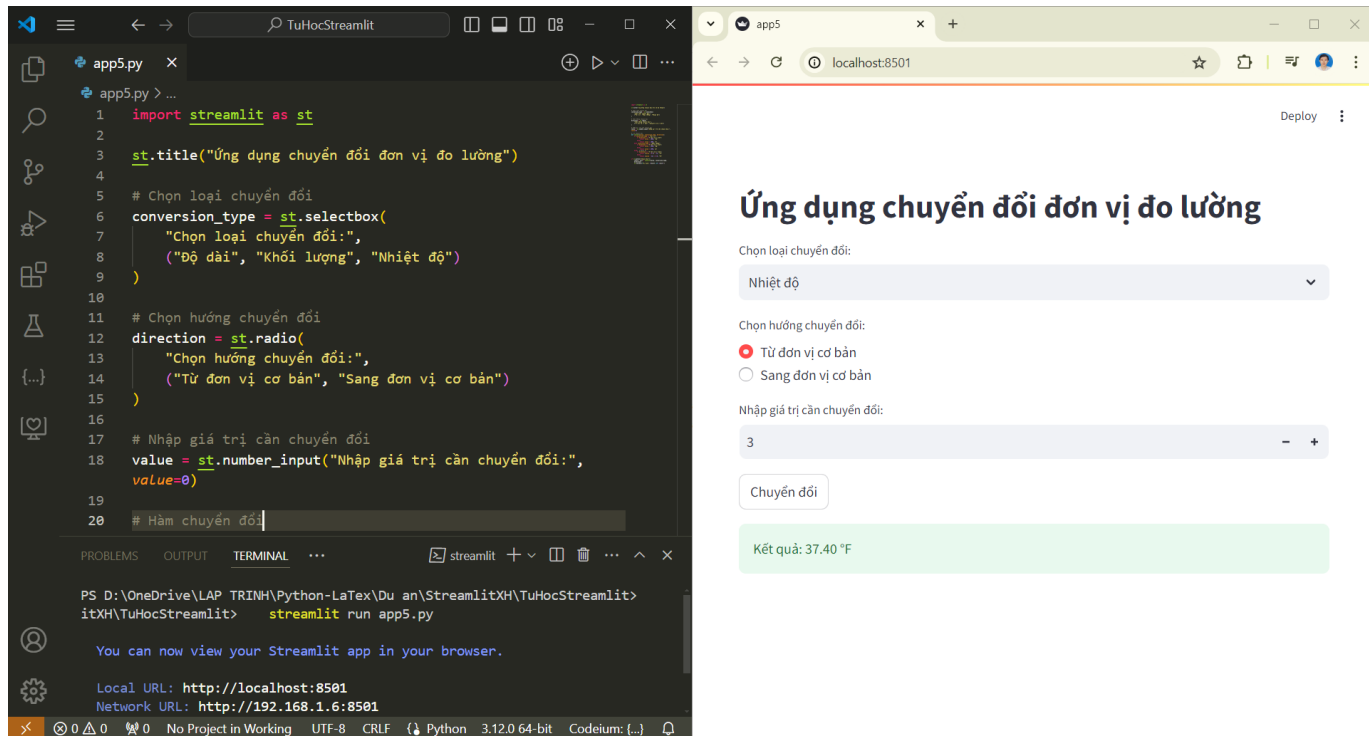
# Hàm chuyển đổi
def convert(value, conversion_type, direction):
    if conversion_type == "Độ dài":
        if direction == "Từ đơn vị cơ bản":
            return value / 1000, "km"
        else:
            return value * 1000, "m"
    elif conversion_type == "Khối lượng":
        if direction == "Từ đơn vị cơ bản":
            return value / 1000, "kg"
        else:
            return value * 1000, "g"
    else: # Nhiệt độ
        if direction == "Từ đơn vị cơ bản":
            return (value * 9/5) + 32, "°F"
        else:
            return (value - 32) * 5/9, "°C"

if st.button("Chuyển đổi"):
    result, unit = convert(value, conversion_type, direction)
    st.success(f"Kết quả: {result:.2f} {unit}")
```

GIẢI THÍCH

- 1-3. Nhập thư viện Streamlit và tạo tiêu đề.
- 5-9. Tạo một selectbox để chọn loại chuyển đổi.
- 11-15. Tạo radio button để chọn hướng chuyển đổi.
- 17-18. Tạo ô nhập số để nhập giá trị cần chuyển đổi.
- 20-36. Định nghĩa hàm convert để thực hiện chuyển đổi.
- 38-40. Tạo nút "Chuyển đổi" và hiển thị kết quả khi nút được nhấn.

MINH HỌA



BÀI TẬP

Hãy mở rộng ứng dụng chuyển đổi đơn vị đo lường bằng cách thêm các tính năng sau:

1. Thêm một loại chuyển đổi mới (ví dụ: chuyển đổi vận tốc giữa km/h và m/s).
2. Cho phép người dùng nhập đơn vị đầu vào và đầu ra cụ thể (ví dụ: chọn giữa m, cm, mm cho độ dài).
3. Thêm một sidebar để hiển thị lịch sử chuyển đổi gần đây.

Gợi ý:

- Sử dụng `st.sidebar` để tạo sidebar.
- Sử dụng một list hoặc dictionary để lưu trữ lịch sử chuyển đổi.
- Sử dụng nhiều `st.selectbox` để cho phép chọn đơn vị đầu vào và đầu ra cụ thể.

KẾT LUẬN

Trong bài học này, chúng ta đã tạo một ứng dụng Streamlit đơn giản để chuyển đổi đơn vị đo lường. Chúng ta đã học cách sử dụng selectbox, radio button và number input để tạo giao diện người dùng, cũng như cách xử lý logic chuyển đổi đơn giản.

Bài học 6

Tạo ứng dụng trực quan hóa dữ liệu đơn giản

MỤC TIÊU

- Học cách tải và xử lý dữ liệu từ file CSV
- Sử dụng Pandas để xử lý dữ liệu
- Tạo biểu đồ đơn giản với Plotly Express
- Hiển thị dữ liệu dưới dạng bảng

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit
- Thư viện Pandas (`pip install pandas`)
- Thư viện Plotly Express (`pip install plotly`)

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st
import pandas as pd
import plotly.express as px

st.title("Ứng dụng trực quan hóa dữ liệu đơn giản")

# Tải dữ liệu
uploaded_file = st.file_uploader("Chọn file CSV", type="csv")
if uploaded_file is not None:
    data = pd.read_csv(uploaded_file)
    st.write("Dữ liệu đã tải:")
    st.write(data.head())

# Chọn cột để trực quan hóa
numeric_columns = data.select_dtypes(include=['float64', 'int64']).columns
x_axis = st.selectbox("Chọn trục X", options=numeric_columns)
y_axis = st.selectbox("Chọn trục Y", options=numeric_columns)

# Vẽ biểu đồ
fig = px.scatter(data, x=x_axis, y=y_axis, title=f"{y_axis} vs {x_axis}")
st.plotly_chart(fig)

# Hiển thị thống kê cơ bản
st.write("Thống kê cơ bản:")
st.write(data[[x_axis, y_axis]].describe())

# Hiển thị dữ liệu dưới dạng bảng
if st.checkbox("Hiển thị dữ liệu dạng bảng"):
    st.write(data)
```

GIẢI THÍCH

- 1-3. Nhập các thư viện cần thiết.
5. Tạo tiêu đề cho ứng dụng.
- 7-12. Tạo widget để tải file CSV và đọc dữ liệu.
 - Sử dụng `st.file_uploader` để cho phép người dùng tải lên file CSV.

- Nếu có file được tải lên, đọc file bằng `pd.read_csv` và hiển thị 5 dòng đầu tiên của dữ liệu.
- 14-17. Chọn các cột số để trực quan hóa.
- Lọc ra các cột số (kiểu `float64` và `int64`).
 - Sử dụng `st.selectbox` để cho phép người dùng chọn cột cho trục X và Y.
- 19-21. Vẽ biểu đồ phân tán (scatter plot) sử dụng Plotly Express. Hiển thị biểu đồ bằng `st.plotly_chart`.
- 23-25. Hiển thị thống kê cơ bản của các cột đã chọn. Sử dụng phương thức `describe()` của Pandas để tính toán và hiển thị thống kê cơ bản của các cột đã chọn.
- 27-29. Tạo tùy chọn để hiển thị toàn bộ dữ liệu dưới dạng bảng.

MINH HỌA

Ứng dụng trực quan hóa dữ liệu đơn giản

Chọn file CSV

Drag and drop file here
Limit 200MB per file • CSV

bang_diem_lop.csv 0.9KB

Dữ liệu tải:

	Họ và tên	Giới tính	Toán	Văn	Anh văn	Vật lý	Hóa học	Sinh học	Điểm trung bình
0	Nguyễn Văn An	Nam	8.5	7.5	9	8	7.5	8	8.08
1	Trần Thị Bình	Nữ	9	8.5	8	7.5	8.5	9	8.42
2	Lê Hoàng Cường	Nam	7	8	7.5	9	8	7.5	7.83
3	Phạm Thị Dung	Nữ	8	9	8.5	7	9	8.5	8.33
4	Hoàng Văn Em	Nam	6.5	7	8	7.5	7	8	7.33

Chọn trục X
Toán

Chọn trục Y
Điểm trung bình

Điểm trung bình vs Toán

Thống kê cơ bản:

	Toán	Điểm trung bình
count	15	15
mean	7.8667	7.9873
std	0.9537	0.5515
min	6.5	7.25
25%	7	7.455
50%	8	8
75%	8.5	8.46
max	9.5	8.75

☒ Hiển thị dữ liệu dạng bảng

	Họ và tên	Giới tính	Toán	Văn	Anh văn	Vật lý	Hóa học	Sinh học	Điểm trung bình
0	Nguyễn Văn An	Nam	8.5	7.5	9	8	7.5	8	8.08
1	Trần Thị Bình	Nữ	9	8.5	8	7.5	8.5	9	8.42
2	Lê Hoàng Cường	Nam	7	8	7.5	9	8	7.5	7.83
3	Phạm Thị Dung	Nữ	8	9	8.5	7	9	8.5	8.33
4	Hoàng Văn Em	Nam	6.5	7	8	7.5	7	8	7.33
5	Đỗ Thị Phương	Nữ	9.5	8.5	9	8.5	8	9	8.75
6	Ngô Quang Giáp	Nam	7.5	6.5	7	8.5	7.5	7	7.33
7	Bùi Thị Hoa	Nữ	8.5	9.5	8	8	9	8.5	8.58

BÀI TẬP

Hãy mở rộng ứng dụng trực quan hóa dữ liệu bằng cách thêm các tính năng sau:

1. Thêm tùy chọn để chọn loại biểu đồ (ví dụ: scatter plot, line plot, bar plot).
2. Cho phép người dùng chọn một cột để phân loại dữ liệu (sử dụng màu sắc khác nhau trên biểu đồ).
3. Thêm tính năng lọc dữ liệu dựa trên một điều kiện do người dùng nhập.

Gợi ý:

- Sử dụng `st.radio()` hoặc `st.selectbox()` để chọn loại biểu đồ.
- Sử dụng tham số `color` trong hàm Plotly Express để phân loại dữ liệu.
- Sử dụng `st.text_input()` để nhập điều kiện lọc và `data.query()` để lọc dữ liệu.

KẾT LUẬN

Trong bài học này, chúng ta đã tạo một ứng dụng Streamlit để trực quan hóa dữ liệu đơn giản. Chúng ta đã học cách tải dữ liệu từ file CSV, xử lý dữ liệu với Pandas, tạo biểu đồ với Plotly Express, và hiển thị dữ liệu dưới dạng bảng.

Bài học 7

Tạo ứng dụng giải phương trình và vẽ đồ thị hàm số

MỤC TIÊU

- Sử dụng các widget tương tác nâng cao của Streamlit trong chủ đề toán học
- Tạo ứng dụng giải phương trình và vẽ đồ thị hàm số
- Sử dụng layout linh hoạt để trình bày kết quả

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit
- Thư viện SymPy ([pip install sympy](#))
- Thư viện Matplotlib ([pip install matplotlib](#))

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st
import sympy as sp
import matplotlib.pyplot as plt
import numpy as np

st.title("Ứng dụng giải phương trình và vẽ đồ thị hàm số")

# Chọn loại phương trình
equation_type = st.selectbox("Chọn loại phương trình:", ["Bậc nhất", "Bậc hai"])

# Nhập hệ số
if equation_type == "Bậc nhất":
    a = st.number_input("Nhập hệ số a:", value=1.0)
    b = st.number_input("Nhập hệ số b:", value=0.0)
else:
    a = st.number_input("Nhập hệ số a:", value=1.0)
    b = st.number_input("Nhập hệ số b:", value=0.0)
    c = st.number_input("Nhập hệ số c:", value=0.0)

# Giải phương trình
x = sp.Symbol('x')
if equation_type == "Bậc nhất":
    equation = sp.Eq(a*x + b, 0)
else:
    equation = sp.Eq(a*x**2 + b*x + c, 0)

solution = sp.solve(equation)

# Hiển thị kết quả
st.subheader("Kết quả:")
col1, col2 = st.columns(2)
with col1:
    st.write("Phương trình:")
    st.latex(sp.latex(equation))
with col2:
    st.write("Nghiem:")
    st.write(solution)

# Vẽ đồ thị
st.subheader("Đồ thị hàm số:")
x_range = np.linspace(-10, 10, 1000)
```

```

if equation_type == "Bậc nhất":
    y = a * x_range + b
else:
    y = a * x_range**2 + b * x_range + c

fig, ax = plt.subplots()
ax.plot(x_range, y)
ax.axhline(y=0, color='r', linestyle='--')
ax.axvline(x=0, color='r', linestyle='--')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.set_title('Đồ thị hàm số')
st.pyplot(fig)

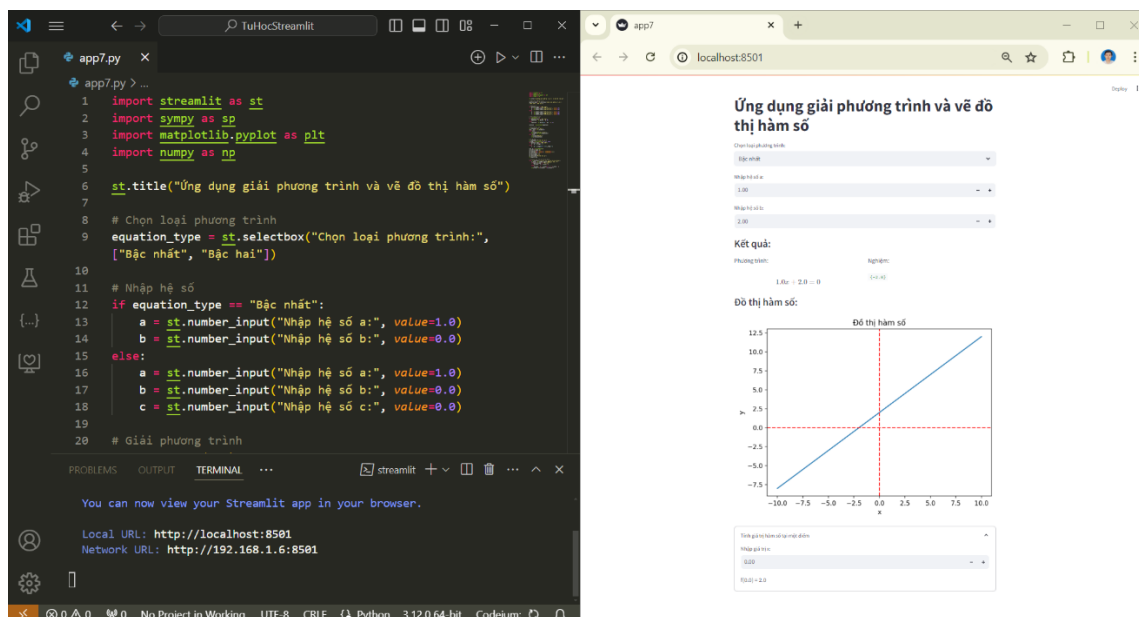
# Tính giá trị hàm số tại một điểm
with st.expander("Tính giá trị hàm số tại một điểm"):
    x_value = st.number_input("Nhập giá trị x:")
    if equation_type == "Bậc nhất":
        y_value = a * x_value + b
    else:
        y_value = a * x_value**2 + b * x_value + c
    st.write(f"f({x_value}) = {y_value}")

```

GIẢI THÍCH

- P1. Chọn loại phương trình: Sử dụng `st.selectbox()` để chọn giữa phương trình bậc nhất và bậc hai.
- P2. Nhập hệ số: Sử dụng `st.number_input()` để nhập các hệ số của phương trình.
- P3. Giải phương trình: Sử dụng thư viện SymPy để giải phương trình một cách chính xác.
- P4. Hiển thị kết quả: Sử dụng `st.columns()` để tạo layout hai cột, hiển thị phương trình và nghiệm.
- P5. Vẽ đồ thị: Sử dụng Matplotlib để vẽ đồ thị hàm số.
- P6. Tính giá trị hàm số: Sử dụng `st.expander()` để tạo một phần có thể mở rộng cho phép người dùng tính giá trị hàm số tại một điểm.

MINH HỌA



Chọn loại phương trình:

Bậc nhất

Nhập hệ số a:

1.00

Nhập hệ số b:

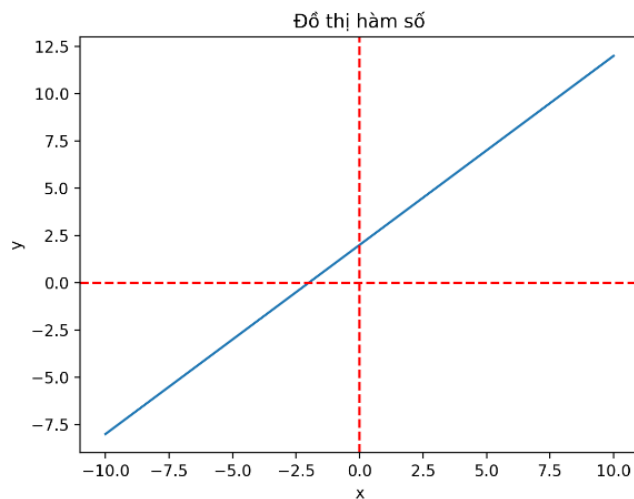
2.00

Kết quả:

Phương trình:

$$1.0x + 2.0 = 0$$

Nghiem:

 $\{-2.0\}$ **Đồ thị hàm số:**

Tính giá trị hàm số tại một điểm

Nhập giá trị x:

0.00

$$f(0.0) = 2.0$$

Chọn loại phương trình:

Bậc hai

Nhập hệ số a:

1.00

Nhập hệ số b:

2.00

Nhập hệ số c:

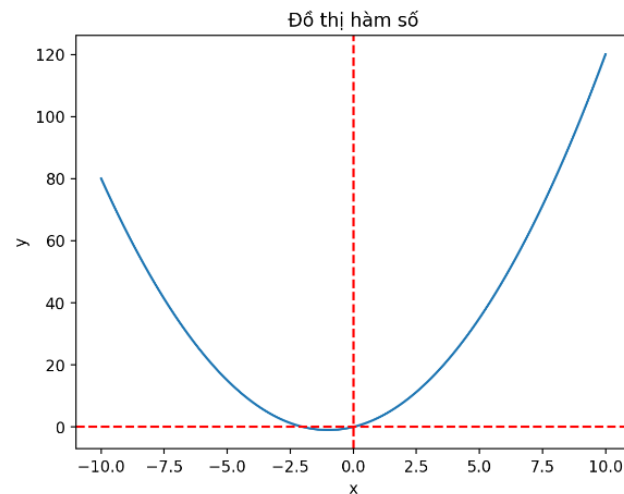
0.00

Kết quả:

Phương trình:

$$1.0x^2 + 2.0x = 0$$

Nghiem:

 $\{-2.0, 0\}$ **Đồ thị hàm số:**

Tính giá trị hàm số tại một điểm

Nhập giá trị x:

0.00

BÀI TẬP

Hãy mở rộng ứng dụng này bằng cách thêm các tính năng sau:

1. Thêm tùy chọn giải phương trình bậc ba.
2. Cho phép người dùng nhập phương trình dưới dạng chuỗi (ví dụ: " $x^2 + 2x + 1 = 0$ ").
3. Thêm tính năng tìm cực trị của hàm số.
4. Cho phép người dùng tùy chỉnh khoảng giá trị x cho đồ thị.

Gợi ý cho bài tập:

1. Để thêm tùy chọn giải phương trình bậc ba, bạn có thể sử dụng `sympy.solve()` với phương trình bậc ba.
2. Để cho phép nhập phương trình dưới dạng chuỗi, sử dụng `st.text_input()` và `sympy.parsing.sympy_parser.parse_expr()` để chuyển đổi chuỗi thành biểu thức SymPy.
3. Để tìm cực trị, sử dụng `sympy.diff()` để tính đạo hàm và `sympy.solve()` để tìm nghiệm của đạo hàm.
4. Để tùy chỉnh khoảng giá trị x, sử dụng `st.slider()` cho phép người dùng chọn giá trị min và max của x.

KẾT LUẬN

Trong bài học này, chúng ta đã tạo một ứng dụng Streamlit để giải phương trình và vẽ đồ thị hàm số. Chúng ta đã sử dụng các widget tương tác nâng cao, kết hợp với các thư viện toán học như SymPy và Matplotlib để tạo ra một ứng dụng hữu ích cho việc học và nghiên cứu toán học.

Bài học 8

Tạo ứng dụng mô phỏng và tính toán xác suất

MỤC TIÊU

- Tạo ứng dụng mô phỏng các thí nghiệm xác suất cơ bản
- Tính toán và hiển thị xác suất lý thuyết và thực nghiệm
- Sử dụng animation để trực quan hóa quá trình mô phỏng
- Áp dụng cache để tối ưu hiệu suất ứng dụng

CÀI ĐẶT

- Python 3.x
- Thư viện Streamlit
- Thư viện Numpy
- Thư viện Plotly

TÁC GIẢ

Trần Lê Xuân Hạnh

EMAIL

xuanhanh123@gmail.com

FACEBOOK

<https://www.facebook.com/hanh.tranlexuan/>

YOUTUBE

<https://www.youtube.com/@tranlexuanhanh2269>

CODE

```
import streamlit as st
import numpy as np
import plotly.graph_objects as go

st.title("Ứng dụng mô phỏng và tính toán xác suất")

# Lựa chọn thí nghiệm
thi_nghiem = st.selectbox("Chọn thí nghiệm:", ["Tung đồng xu", "Tung xúc xắc", "Rút bóng"])

# Các hàm cache cho từng loại thí nghiệm
@st.cache_data
def tung_dong_xu(n):
    return np.random.choice(["Mặt sấp", "Mặt ngửa"], size=n)

@st.cache_data
def tung_xuc_xac(n):
    return np.random.randint(1, 7, size=n)

@st.cache_data
def rut_bong(n, do, xanh):
    return np.random.choice(["Đỏ", "Xanh"], size=n,
                             p=[do/(do+xanh), xanh/(do+xanh)])

# Số lần mô phỏng
so_lan_mo_phong = st.slider("Số lần mô phỏng:", 100, 10000, 1000)

# Lựa chọn thí nghiệm cụ thể và thực hiện mô phỏng
if thi_nghiem == "Tung đồng xu":
    ket_qua = tung_dong_xu(so_lan_mo_phong)
    ket_qua = (ket_qua == "Mặt ngửa").astype(int)
    xac_suat_ly_thuyet = 0.5

elif thi_nghiem == "Tung xúc xắc":
    ket_qua = tung_xuc_xac(so_lan_mo_phong)
    su_kien = st.selectbox("Chọn sự kiện:", ["Tung được số chẵn", "Tung được số lẻ", "Tung được số cụ thể"])
    if su_kien == "Tung được số chẵn":
```

```

ket_qua = (ket_qua % 2 == 0).astype(int)
xac_suat_ly_thuyet = 0.5 # Xác suất tung được số chẵn
elif su_kien == "Tung được số lẻ":
    ket_qua = (ket_qua % 2 != 0).astype(int)
    xac_suat_ly_thuyet = 0.5 # Xác suất tung được số lẻ
else:
    so_cu_the = st.number_input("Chọn số cụ thể:", 1, 6, 1)
    ket_qua = (ket_qua == so_cu_the).astype(int)
    xac_suat_ly_thuyet = 1/6 # Xác suất lý thuyết cho mỗi
    mặt của xúc xắc

else: # Rút bóng
    so_bong_do = st.number_input("Số bóng đỏ:", 1, 100, 50)
    so_bong_xanh = st.number_input("Số bóng xanh:", 1, 100, 50)
    ket_qua = rut_bong(so_lan_mo_phong, so_bong_do,
so_bong_xanh)
    su_kien = st.radio("Chọn sự kiện:", ["Rút được bóng đỏ",
    "Rút được bóng xanh"])
    if su_kien == "Rút được bóng đỏ":
        ket_qua = (ket_qua == "Đỏ").astype(int)
        xac_suat_ly_thuyet = so_bong_do / (so_bong_do +
so_bong_xanh)
    else:
        ket_qua = (ket_qua == "Xanh").astype(int)
        xac_suat_ly_thuyet = so_bong_xanh / (so_bong_do +
so_bong_xanh)

# Tính xác suất thực nghiệm
xac_suat_thuc_nghiem = np.cumsum(ket_qua) / np.arange(1,
len(ket_qua) + 1)

# Tạo đồ thị so sánh xác suất thực nghiệm và lý thuyết
bieu_do = go.Figure()
bieu_do.add_trace(go.Scatter(y=xac_suat_thuc_nghiem,
mode='lines', name='Xác suất thực nghiệm'))
bieu_do.add_trace(go.Scatter(y=[xac_suat_ly_thuyet] *
len(xac_suat_thuc_nghiem), mode='lines', name='Xác suất lý
thuyết'))
bieu_do.update_layout(title='Xác suất thực nghiệm vs Xác suất lý
thuyết',
                        xaxis_title='Số lần thử',
                        yaxis_title='Xác suất')
st.plotly_chart(bieu_do)

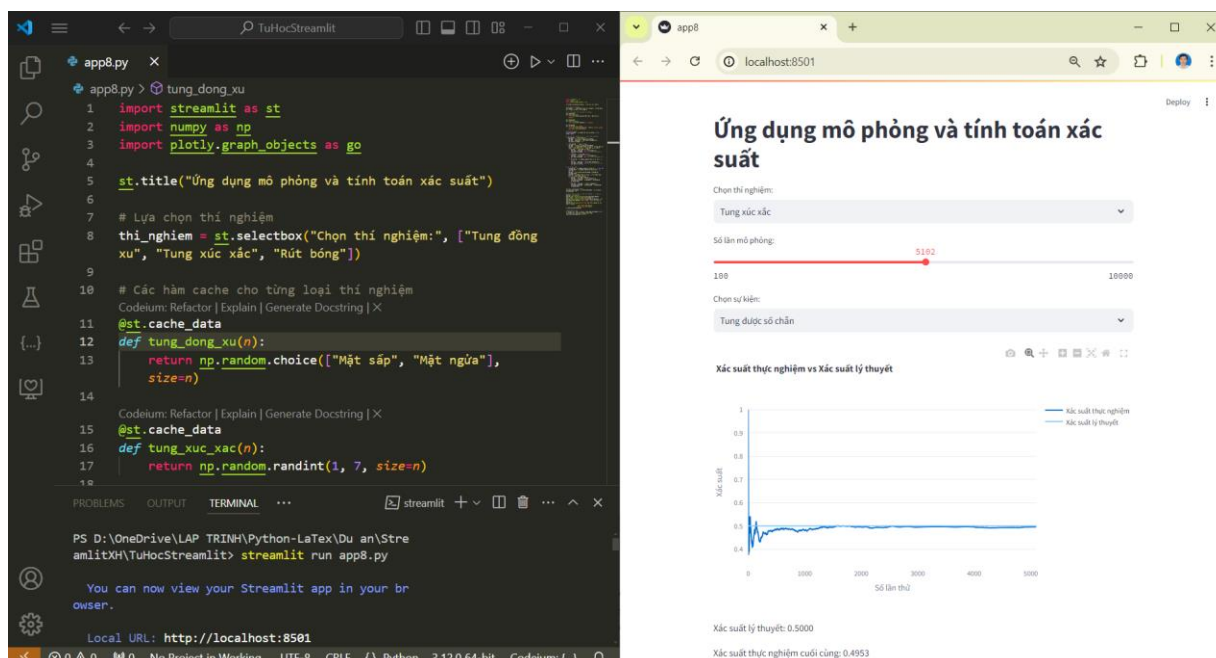
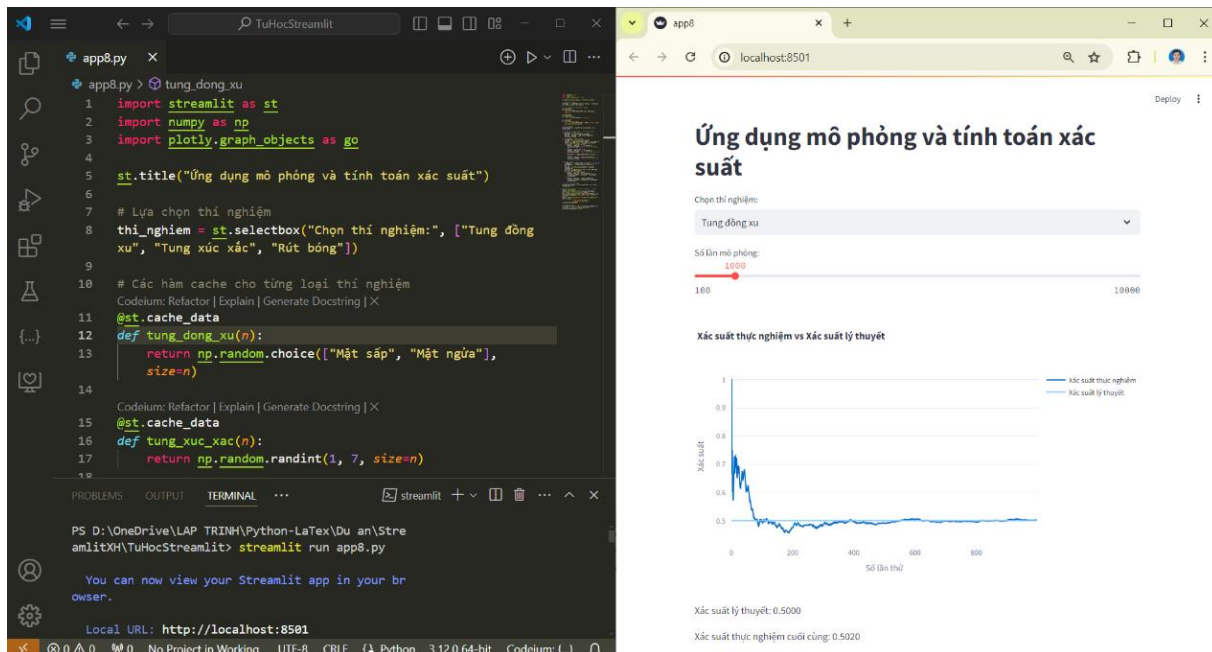
# Hiển thị kết quả
st.write(f"Xác suất lý thuyết: {xac_suat_ly_thuyet:.4f}")
st.write(f"Xác suất thực nghiệm cuối cùng:
{xac_suat_thuc_nghiem[-1]:.4f}")

```

GIẢI THÍCH

- P1. Chọn thí nghiệm: Sử dụng `st.selectbox()` để chọn giữa các thí nghiệm xác suất khác nhau.
- P2. Các hàm mô phỏng: Sử dụng `@st.cache_data` để cache kết quả mô phỏng, tối ưu hiệu suất.
- P3. Tùy chỉnh thí nghiệm: Cho phép người dùng chọn số lần mô phỏng và các tham số khác tùy thuộc vào thí nghiệm.
- P4. Tính toán xác suất: Tính xác suất lý thuyết và thực nghiệm cho mỗi thí nghiệm.
- `xac_suat_thuc_nghiem = np.cumsum(ket_qua) / np.arange(1, len(ket_qua) + 1)`: Tính xác suất thực nghiệm tích lũy.
- P5. Vẽ biểu đồ: Sử dụng Plotly để vẽ biểu đồ so sánh xác suất thực nghiệm và lý thuyết.
- `bieu_do.add_trace(...)`: Thêm đường cho xác suất thực nghiệm và lý thuyết.
 - `bieu_do.update_layout(...)`: Cập nhật tiêu đề và nhãn trục cho biểu đồ.

MINH HỌA



Ứng dụng mô phỏng và tính toán xác suất

Chọn thí nghiệm:

Tung xúc xắc

Số lần mô phỏng:

1000

100 10000

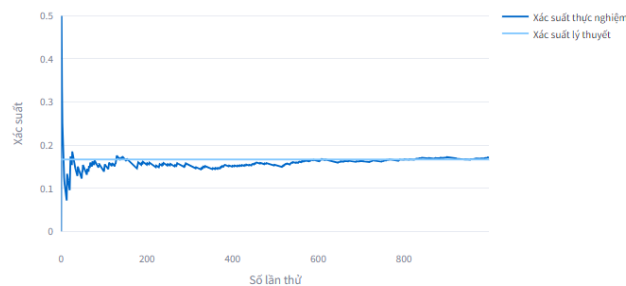
Chọn sự kiện:

Tung được số cụ thể

Chọn số cụ thể:

2

Xác suất thực nghiệm vs Xác suất lý thuyết



Xác suất lý thuyết: 0.1667

Xác suất thực nghiệm cuối cùng: 0.1710

Chọn thí nghiệm:

Rút bóng

Số lần mô phỏng:

2050

100 10000

Số bóng đỏ:

40

Số bóng xanh:

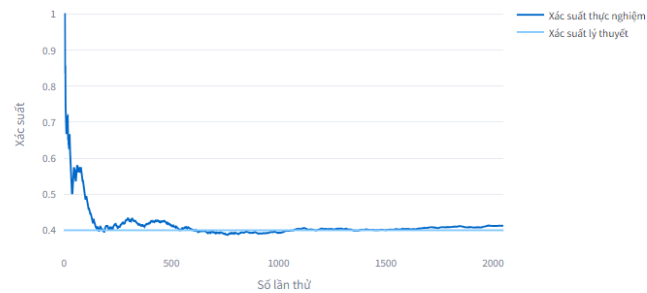
60

Chọn sự kiện:

☒ Rút được bóng đỏ

☐ Rút được bóng xanh

Xác suất thực nghiệm vs Xác suất lý thuyết



Xác suất lý thuyết: 0.4000

Xác suất thực nghiệm cuối cùng: 0.4122

BÀI TẬP

Hãy mở rộng ứng dụng này bằng cách thêm các tính năng sau:

1. Thêm thí nghiệm xác suất mới (ví dụ: rút bài từ bộ bài 52 lá).
2. Cho phép người dùng thực hiện nhiều lần mô phỏng và hiển thị phân phối của kết quả.
3. Thêm tính năng tính toán khoảng tin cậy cho xác suất thực nghiệm.
4. Tạo animation để hiển thị quá trình hội tụ của xác suất thực nghiệm.
5. Thêm tùy chọn để so sánh nhiều thí nghiệm xác suất trên cùng một biểu đồ.

Gợi ý:

1. Sử dụng `np.random.choice()` với tham số `replace=False` để mô phỏng rút bài không hoàn lại.
2. Sử dụng `st.button()` để kích hoạt nhiều lần mô phỏng và `plotly.graph_objects.Histogram` để vẽ histogram.
3. Sử dụng công thức khoảng tin cậy cho tỷ lệ trong phân phối nhị thức.
4. Sử dụng `st.empty()` và `time.sleep()` để tạo animation đơn giản.
5. Sử dụng `st.multiselect()` để chọn nhiều thí nghiệm và thêm nhiều traces vào biểu đồ Plotly.

KẾT LUẬN

Trong bài học cuối cùng này, chúng ta đã tạo một ứng dụng Streamlit để mô phỏng và tính toán xác suất. Chúng ta đã học cách sử dụng các widget tương tác, tạo biểu đồ động với Plotly, và áp dụng cache để tối ưu hiệu suất. Ứng dụng này minh họa cách Streamlit có thể được sử dụng để tạo ra các công cụ giáo dục tương tác trong lĩnh vực xác suất và thống kê.